

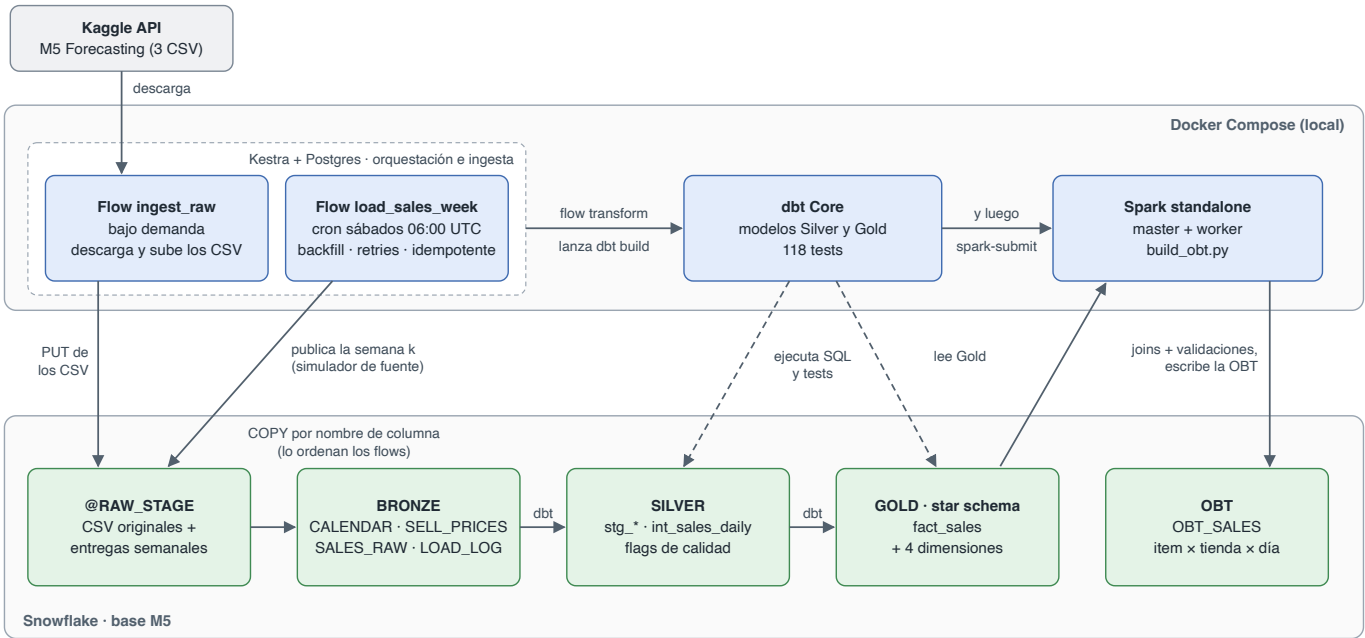
PSet 2 — Memo técnico: pipeline ELT de M5

4 de octubre de 2026 · Repositorio: github.com/rosvjames/M5_forecast_pipeline

El pipeline lleva las 278 semanas de M5 desde Kaggle hasta una OBT de 59.181.090 filas en Snowflake, con 133 tests de dbt en verde y Kestra orquestando la cadena completa. Alimenta la pregunta del proyecto: ¿cuánto stock de seguridad se necesita para alcanzar un nivel de servicio objetivo usando nuestro pronóstico, frente a los baselines?

1. Arquitectura

Snowflake es el único destino de los datos. Kestra, dbt y Spark corren en contenedores locales definidos en `docker-compose.yml`, con versiones fijas, y solo envían SQL o mueven datos por conector. Cada capa vive en su propio esquema de la base M5 (`BRONZE`, `SILVER`, `GOLD`, `OBT`) porque cada una tiene un solo escritor (Kestra, dbt, dbt y Spark) y permisos distintos: dbt no puede pisar el dato original de Bronze, y quien consume lee solo de `GOLD` o de `OBT`.



Flechas sólidas verticales: movimiento de datos. Punteadas: SQL que dbt ejecuta dentro de Snowflake. Horizontales entre herramientas: orquestación de Kestra.

- Kestra 1.3.40 (+ Postgres 16):** `ingest_raw` descarga M5 con la API de Kaggle, sube los 3 CSV a `@RAW_STAGE` y carga `CALENDAR` y `SELL_PRICES`. `load_sales_week` publica y carga una entrega semanal de ventas. `transform` se dispara cuando esa carga termina en `SUCCESS` y ejecuta dbt y luego Spark.
- dbt Core 1.9 (contenedor de un solo uso):** construye Silver (limpieza y flags) y Gold (star schema) con `source()` y `ref()`, y corre los tests.
- Spark 4.1.3 standalone (1 master, 1 worker de 4 GB):** lee Gold con el conector `spark-snowflake 3.2.2`, hace los joins, valida y escribe `OBT.OBT_SALES`.

Orquestación de punta a punta. `transform` tiene un trigger de tipo Flow sobre `load_sales_week`. Primero consulta si Bronze está al día (todas las semanas hasta la que toca según el reloj simulado): así un backfill de 278 cargas dispara una sola reconstrucción. Luego corre `dbt build` en un contenedor con la imagen de dbt; si un test falla, el flow falla y Spark no corre. Por último lanza `spark-submit` en el master. Tiene `concurrency: 1`, timeouts y un reintento por paso; los dos pasos son idempotentes.

El pipeline se autentica con un usuario de servicio y key-pair, bajo el rol `M5_PIPELINE` de mínimo privilegio. Ningún secreto está en el repositorio: todo sale de `.env`.

2. Ingesta

Bronze tiene las 278 entregas semanales de ventas (8.476.220 filas en `SALES_RAW`), 6.841.121 precios y 1.969 días de calendario, cargados por flows de Kestra sin SQL escrito a mano.

Aspecto	Implementación
---------	----------------

Fuente	M5 Forecasting (Walmart) por la API de Kaggle: <code>sales_train_evaluation</code> , <code>sell_prices</code> y <code>calendar</code> . <code>sales_train_validation</code> es un prefijo exacto de <code>evaluation</code> y no se ingiere.
Granularidad	Una fila por serie (item × tienda) y entrega semanal, con 7 columnas <code>d_N</code> : 30.490 filas por semana. Precios por tienda × item × semana Walmart.
Frecuencia	Trigger Schedule <code>0 6 * * 6</code> (sábados 06:00 UTC), <code>concurrency: 1</code> con cola.
Retries	Exponencial por tarea: 10 s inicial, máximo 2 min, 3 intentos. Timeout de 10 min en la carga y 15 min en la descarga.
Errores	Bloque <code>errors</code> : log de nivel ERROR y fila <code>FAILED</code> con el mensaje en <code>BRONZE.LOAD_LOG</code> .
Backfill	Nativo del trigger Schedule, desde 2021-06-12: 277 ejecuciones, 277 en SUCCESS, 0 reintentos, 2 h 44 min (mediana 27,5 s por semana).

Reloj simulado. M5 es un archivo estático, así que cada tick semanal se asigna a una semana de M5: k = semanas completas entre el ancla (sábado 2021-06-12) y la fecha del trigger. La semana k trae $d_{(7k+1)}$ a $d_{(7k+7)}$; la 277 solo trae d_{1940} y d_{1941} . Fuera de 0–277 el flow no carga nada.

Proceso de carga. La tarea `publish_week` hace de fuente: publica en el stage un CSV ancho con la jerarquía y los 7 días de la semana. La tarea `load_week` lo copia a `SALES_RAW` por nombre de columna (`MATCH_BY_COLUMN_NAME` + schema evolution) y agrega `week_idx`, `_batch_id`, `_loaded_at`, `_source_file` y `_source_row`. No transforma ningún valor.

Idempotencia. `load_week` ejecuta `DELETE` de la semana, `COPY`, `UPDATE` de metadatos y registro en `LOAD_LOG`. `COPY`, `UPDATE` y registro son siempre atómicos; en una recarga el `DELETE` entra en la misma transacción. Re-ejecutar una semana la reemplaza sin duplicar.

Prueba de fallos. Con el input `simulate_failure`, un fallo transitorio se recupera en el segundo intento. Uno permanente agota los 3 intentos, Snowflake revierte el `DELETE` las tres veces y la semana queda completa.

Decisión revisada. La primera versión hacía el `UNPIVOT` en la ingesta y leía columnas por posición: un cambio de formato en la fuente habría corrompido datos sin error. Tras el feedback del profesor, Bronze guarda la entrega tal cual y el paso a formato largo se hace en dbt. La versión nueva reproduce las mismas filas y la misma suma de ventas que la anterior.

3. Data Quality

El 21,0 % de las filas de ventas no representa demanda y se marca con flags en vez de borrarse: así la malla serie × día queda completa y cada exclusión es explícita y reversible. La calidad se midió antes de limpiar con 42 chequeos en SQL (`dbt/analyses/dq_01` a `dq_04`) sobre Bronze y sobre `stg_sales`, que solo cambia la forma de las ventas y no sus valores. Cifras al 4-oct-2026, con 59.181.090 días-serie.

Dimensión	Problema	Evidencia	Acción en Silver	Justificación
Complejidad	Ceros antes del lanzamiento del producto	12.299.413 días-serie: 20,8 % de la matriz y 30,6 % de todos los ceros	Flag <code>is_pre_launch</code> (antes de la primera semana con precio). Se excluyen de entrenamiento y métricas	El producto no estaba en la tienda: el cero no es demanda. Hay 0 ventas antes del lanzamiento
Precisión	Tiendas cerradas en Navidad	Los cinco 25-dic suman 78 unidades (11 a 20 por día), frente a una mediana diaria de 33.830	Flag <code>is_christmas_closed</code> . Se excluyen	El cero lo causa el cierre. Si el modelo lo aprende, subestima la víspera y el año siguiente
Precisión	Cierres puntuales de tienda	2 días-tienda de 19.410 bajo el 10 % de su mediana móvil: <code>WI_1</code> 2011-02-02 (2 u) y <code>TX_2</code> 2015-03-24 (131 u)	Flag <code>is_store_closed</code> desde el <code>seed store_closures</code>	Mismo motivo, en una sola tienda. Otras 19 caídas bajo el 50 % (12 en Thanksgiving) son demanda real y no se marcan
Precisión	Quiebres de stock no observados	M5 no trae inventario y el precio no los delata (0 de 46.881.677 días sin precio). Rachas de ceros anómalas: 1.946.313 días (4,2 % de los días activos) con la regla conservadora y 6.778.366 (14,5 %) con la binomial negativa	Flags <code>is_suspected_stockout_conservative</code> (tasa ≥ 1 u/día y racha ≥ 14 días) e <code>is_suspected_stockout_nb</code> (racha ≥ 7 días, $p < 0,001$)	Una racha de ceros no distingue quiebre de caída real de demanda: se marca con dos reglas para medir la sensibilidad, sin borrar ni imputar
Precisión	Picos extremos de venta	6.513 días-serie (0,011 %) en 1.938 series (6,4 %): más de 10× la mediana de días con venta y al menos 20 u	Flag <code>is_sales_spike</code> , solo diagnóstico. No se eliminan	Mezclan compras grandes reales con posibles errores (601 u en una serie de mediana 1). Borrarlos quitaría demanda real

Precisión	<code>sell_price</code> es un promedio semanal, no el precio de lista	21,3 % de los precios termina en centavos atípicos (2, 3, 4, 6)	Se conserva y se documenta	Es el único precio disponible; sirve como feature y para valorizar inventario
Precisión	Precios extremos	445 precios (0,007 %) fuera de 0,2x a 5x la mediana de su serie; 12 precios de \$0,01	Se conservan	Corregirlos exigiría inventar un valor y no mueven el ingreso agregado
Complejidad	Nulos en eventos	1.807 de 1.969 días sin evento (91,8 %); nombre y tipo siempre nulos a la vez	Sin imputación; se deriva <code>is_event_day</code>	Nulo estructural: día sin evento

Consistencia y validez: sin hallazgos. No hay llaves duplicadas en ventas ni en precios, y la jerarquía no tiene contradicciones en las 30.490 series. Los mismos 30.490 pares `item` × `tienda` aparecen en ventas y en precios. Las ventas son enteros no negativos, los precios son positivos y la conversión de `d` a fecha es exacta en los 1.969 días.

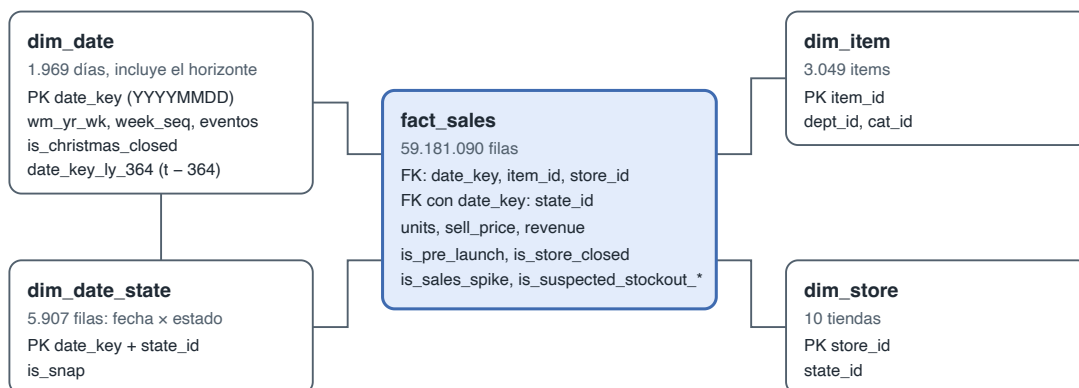
Los flags y la fuga de información. `is_pre_launch` e `is_christmas_closed` se conocen el mismo día y pueden filtrar el entrenamiento. `is_sales_spike` usa la mediana de la serie completa: no es feature, no filtra y no excluye días de la métrica. Los flags de quiebre calculan la tasa y la dispersión solo con días anteriores a la racha, pero marcan la racha completa, cuyo largo se conoce al terminar: filtran rachas cerradas y no son feature.

4. Transformaciones y modelado

dbt construye Silver y Gold con 10 modelos y un seed, enlazados con `source()` y `ref()`. Los 133 tests del proyecto pasan sobre los datos actuales.

Capa	Objetos	Qué hace
Bronze	SALES_RAW, SELL_PRICES, CALENDAR, LOAD_LOG	Conserva el dato original con metadatos de carga. Solo se testea como contrato de ingesta: llaves naturales únicas y no nulas.
Silver	<code>stg_sales</code>	Pasa las ventas de ancho a largo por nombre de columna, sin lista escrita a mano. Incremental por entrega semanal (<code>delete+insert</code> por <code>week_idx</code>).
Silver	<code>stg_calendar</code> , <code>stg_sell_prices</code>	Tipos, SNAP booleano, eventos y semana de lanzamiento de cada serie.
Silver	<code>int_sales_daily</code>	Une ventas, calendario y precios por tienda + <code>item</code> + semana Walmart; calcula ingreso, SNAP del estado y los flags de calidad.
Silver	<code>int_stockout_flags</code>	Rachas de ceros por serie y las dos reglas de quiebre sospechoso. Solo guarda los días marcados (6,8 M filas); el hecho los trae con <code>left join</code> .
Gold	<code>fact_sales</code> y 4 dimensiones	Star schema, tablas reconstruidas en cada corrida.

Grain del hecho. `fact_sales` tiene una fila por `item` × `tienda` × `día` con venta publicada: 30.490 series × 1.941 días = 59.181.090 filas. `units` y `revenue` son aditivas. `sell_price` no lo es, y es nulo exactamente en las filas previas al lanzamiento.



Star schema de Gold: un hecho y cuatro dimensiones, todas N:1. El SNAP usa la llave compuesta fecha + estado.

- **SNAP aparte.** Depende de la fecha y del estado, así que no cabe en `dim_date` ni en `dim_store`. El hecho lleva `state_id` como mitad de esa llave, y un test verifica que coincide con el estado de la tienda.
- **dim_date cubre el horizonte.** Incluye los 28 días futuros (`d_1942` a `d_1969`), un índice secuencial de semana (el código de semana Walmart no se puede restar: el año fiscal 2013 tiene 53 semanas) y la llave del mismo día del año anterior (`t - 364`),

base de uno de los baselines.

- **Llaves naturales.** M5 es estático y sus atributos no cambian en el tiempo; no hacen falta llaves sustitutas.

Tests con reglas del negocio: grain único, relaciones del hecho a las 4 dimensiones, mismo conteo entre Silver y Gold, ninguna venta antes del lanzamiento, ninguna venta sin precio, ninguna fila en el horizonte, calendario sin huecos, SNAP completo por fecha × estado y semanas completas tras el paso a formato largo.

5. Spark y OBT

OBT.OBT_SALES tiene 59.181.090 filas y 40 columnas al mismo grain del hecho, item × tienda × día, y coincide con fact_sales en conteo, unidades (66.927.173) e ingreso.

Construcción. spark/build_obt.py lee fact_sales y las 4 dimensiones con el conector, con autopushdown apagado para que el join lo haga Spark y no Snowflake. Usa left joins con broadcast de las dimensiones (la mayor tiene 5.907 filas): el hecho no se redistribuye por la red, y una fila sin dimensión quedaría con nulos en vez de desaparecer. Escribe con tabla de staging e intercambio atómico; si falla a mitad, la OBT anterior queda intacta. La corrida tarda unos 12,5 min con un worker de 2 núcleos y 4 GB.

Contenido. Objetivo (units), precio e ingreso, jerarquía de item y tienda, calendario completo (eventos, índice de semana, llave t – 364), SNAP del estado, flags de calidad y linaje. Los lags y medias móviles no se materializan: dependen del corte de backtesting, y calcularlos aquí con la serie completa sería fuga de información.

Validación de los joins. Todas corren antes de escribir; si una falla, el job termina con error y no escribe.

Validación	Resultado
Filas de la OBT frente a filas del hecho	59.181.090 = 59.181.090
Llaves (item, tienda, fecha) repetidas	0
Nulos en 9 columnas que vienen de dimensiones	0
Filas con estado del hecho distinto al de dim_store	0
Filas releídas de Snowflake tras escribir	59.181.090

Como control independiente, el mismo join hecho en SQL dentro de Snowflake da un resultado idéntico en conteos, sumas y hash de 11 columnas.

Cuándo usar cada modelo en este proyecto.

- **Star schema:** análisis y reportes por nivel de la jerarquía (error por departamento, tienda o total), tests de integridad, y cambios baratos: agregar un atributo toca una dimensión de hasta 3.049 filas, no 59 millones. También es la fuente del calendario y el SNAP del horizonte futuro.
- **OBT:** entrenar y evaluar el modelo. Es una sola lectura ancha sin joins en el código de modelado; los lags se calculan por serie, el backtesting filtra por día y las exclusiones son filtros de fila. Su costo es la redundancia y una reescritura completa con cada cambio.

6. Batch vs. streaming

Un lote semanal es suficiente: la decisión que alimenta el pipeline se toma una vez por semana y la cadena completa, de la carga a la OBT, tarda unos 15 minutos (1 min de carga, 1,5 de dbt y 12 de Spark).

- **Frecuencia de los datos.** La fuente entrega una observación por serie y día, y precios por semana. No emite eventos.
- **Latencia que pide el negocio.** El stock de seguridad y el punto de reorden se recalculan cada semana, con horizonte de 28 días y lead time supuesto de 7 días. Un dato con horas de retraso no cambia ninguno de los dos.
- **Métricas sobre ventanas cerradas.** El WRMSSE, el error acumulado del lead time y su percentil se calculan por corte de backtesting, no por evento.
- **Reproceso simple.** Corregir una semana es recargarla: la carga y el modelo incremental reemplazan la semana completa.
- **Costo.** Un warehouse XS con auto-suspend de 60 s trabaja minutos por semana, frente a cómputo siempre encendido.

Streaming se justificaría si cambiara el caso de uso: revisión continua del inventario contra el punto de reorden, con ventas por transacción e inventario en línea (M5 no trae inventario); reposición dentro del día, por ejemplo perecibles con lead time menor a un día; alertas de quiebre mientras ocurre la racha de ceros; o precios y promociones que cambien dentro del día. En ese caso la fuente sería un flujo de eventos (Kafka o Snowpipe Streaming), Bronze un append de eventos y Silver tendría modelos incrementales continuos. El reentrenamiento del modelo seguiría en batch.

7. Limitaciones

Datos

- **Las ventas no son demanda.** Los flags de quiebre son una aproximación: marcan entre 4,2 % y 14,5 % de los días activos y mezclan quiebres, descatalogaciones y caídas reales de demanda, que M5 no permite separar. Como cota superior, las ventas perdidas serían 8,8 % a 12,8 % de las unidades. En el holdout (d_1914 a d_1941) solo marcan el 4,0 % de los días, porque el fin de los datos corta las rachas. El nivel de servicio del backtest se mide contra ventas y será optimista; los flags permiten acotar cuánto. Se revisaron 13 datasets con inventario o quiebres observados y ninguno cumplía los requisitos (al menos 2 años diarios, unidades y precios reales, licencia usable), así que se mantiene M5.
- **La OBT llega hasta d_1941.** Para pronosticar d_1942 a d_1969 hay que armar la malla desde `dim_date`, `dim_date_state` y los precios futuros, que hoy solo están en Silver. Tampoco trae clase ABC ni de intermitencia: dependen del corte y se calculan aguas abajo.
- **Columnas que no pueden ser features.** `is_sales_spike` usa la mediana de la serie completa, los flags de quiebre usan el largo final de la racha y `revenue` se deriva del objetivo. Viajan en la OBT solo para diagnóstico, filtros y ponderación.
- **Precio.** `sell_price` es un promedio semanal de venta, no un costo: valorizar el inventario con él sobrestima el capital inmovilizado.
- **Cierres de tienda.** Salen de un seed manual con 2 casos; uno nuevo no se detecta solo.
- **Alcance.** Un solo holdout con valores reales (d_1914 a d_1941), datos de 2011 a 2016 y solo de EE. UU.

Arquitectura

- **Orquestación atada al host.** `transform` lee el proyecto dbt de un volumen local y lanza Spark con `docker exec` por el socket de Docker. Sirve para este Docker Compose; con un Kestra remoto habría que clonar el repositorio y usar un cluster con API de envío de jobs.
- **El tick semanal llegó tarde.** El tick del 2026-10-03 06:00 UTC no se ejecutó a su hora (portátil suspendido) y `recoverMissedSchedules` no lo recuperó hasta que el flow se volvió a guardar el 4-oct; entretanto la semana 277 se había cargado por la API. Ese día el trigger disparó, recargó la semana sin duplicar y encadenó `transform`: 128 nodos de dbt en verde y la OBT validada. La causa del retraso sigue sin identificar.
- **Cluster frágil ante el reposo del equipo.** En esa corrida el worker de Spark había perdido el registro en el master y el job esperó recursos 28 min hasta reiniciarlo. `transform` ahora revisa que haya un worker vivo y lo reinicia antes de enviar el job.
- **Fuente simulada.** `publish_week` genera cada entrega desde el archivo estático y lo lee por posición. Precios y calendario se cargan completos de una vez, sin reloj.
- **Bronze ancha.** `SALES_RAW` tiene 1.952 columnas por cómo publica M5. El `UPDATE` posterior al `COPY` recorre una tabla que crece: la carga pasó de 12 s a 103 s por semana y obliga a `concurrency: 1`. Mejora: derivar `week_idx` del nombre del archivo.
- **Reconstrucciones completas.** Solo `stg_sales` es incremental; `int_sales_daily`, los flags de quiebre, el hecho y la OBT se rehacen enteros en cada carga semanal.
- **Spark local.** Con 4 GB no se puede cachear el hecho, así que cada validación relee Snowflake. Validación y escritura no comparten lectura: se asume que Gold no se reconstruye durante el job.
- **Un solo entorno.** Cuenta trial de Snowflake, sin separación entre desarrollo y producción.